Chapter 13

Security Essentials

Why this chapter matters: a design that is fast, scalable and available but leaks user data or lets anyone call an admin API is a failed design. Security is a **property of the whole system** — edge, services, data and operations — and must be considered from the first diagram, not bolted on. Your original notes list the four essentials: **encrypt in transit and at rest**, **sanitize inputs against XSS and SQL injection**, **use parameterized queries**, and apply the **principle of least privilege**. This chapter makes each of them concrete and adds what senior engineers are expected to know: authentication and authorization, tokens and OAuth, secrets, the OWASP lists, and secure-by-design thinking.

⚡ **QUICK RECALL — Chapter map**

- 13.1 Security mindset & threat modelling → 13.2 Cryptography basics → 13.3 Encrypt in transit and at rest (keys, KMS)
- 13.4 Authentication (passwords, MFA, sessions, JWT, OAuth2/OIDC) → 13.5 Authorization (RBAC/ABAC, IDOR, least privilege)
- 13.6 Input handling: SQL injection, XSS, CSRF, SSRF… → 13.7 OWASP Top 10 → 13.8 API & web checklist
- 13.9 Infrastructure, secrets, supply chain, DDoS → 13.10 Logging, detection, compliance → 13.11 Security in design interviews
- 13.12 Worked examples → 13.13 Practice, Q&A, cheat sheet

✎ **CORRECTION TO YOUR ORIGINAL NOTE — On your original note**

- The source note says that, unless you have a security background, *“you probably won’t need to know more than the basics”*. For junior roles that is fair; for **senior system design** you are expected to reason about authentication, authorization, token design, secrets, and attack surface for every component you draw. Also refine “sanitize all user inputs”: the strongest defences are **parameterised queries** (SQL injection) and **context-specific output encoding** (XSS); input validation with allow-lists helps, but ‘sanitizing’ strings alone is not a reliable defence.

13.1 The security mindset & threat modelling

Security protects the **CIA triad**: **Confidentiality** (only authorised parties can read data), **Integrity** (data and code cannot be altered undetected), and **Availability** (the system keeps working — DDoS and ransomware attack this).

Principle	Meaning	Example
Least privilege	Every user, service and process gets only the permissions it needs, for only as long as it needs them	The order service’s DB user can read/write its own tables, not DROP or read billing
Defence in depth	Multiple independent layers, so one failure isn’t fatal	CDN/WAF → gateway auth → service authZ → DB roles → encryption
Zero trust	Never trust a request because of where it comes from; authenticate and authorise every call, even inside the network	mTLS between services, per-request tokens
Secure by default / fail closed	Defaults are safe; on error deny access	Auth service down → reject (or cached deny), not allow
Minimise attack surface	Expose less: fewer endpoints, ports, dependencies, privileges	Private subnets, close unused ports, remove debug endpoints
Never trust input	Everything from clients, third parties and even other services is untrusted until validated	Validate, encode, parameterise

Principle	Meaning	Example
Don’t roll your own crypto	Use vetted libraries and standard protocols	TLS, AES-GCM, Argon2 via maintained libraries

Threat modelling with STRIDE — ask for each component and data flow: what can go wrong?

Threat	Violates	Example	Typical mitigation
Spoofing identity	Authentication	Stolen password, forged token, fake webhook	MFA, signed tokens, mTLS, webhook signatures
Tampering	Integrity	Modified request, altered data at rest/in transit	TLS, HMAC/signatures, DB permissions, checksums
Repudiation	Non-repudiation	“I never made that transfer”	Audit logs, signed events, immutable storage
Information disclosure	Confidentiality	Leaked backup, verbose errors, IDOR	Encryption, authorisation, least-privilege, safe errors
Denial of service	Availability	L7 flood, expensive query, queue flooding	Rate limits, CDN/WAF, autoscaling, back pressure (Chapter 12)
Elevation of privilege	Authorization	User becomes admin; SQLi to read others’ data	Server-side authZ checks, input handling, sandboxing

◆ **SENIOR INSIGHT** — **Do threat modelling at design time**

- Draw a data-flow diagram, mark **trust boundaries** (internet ↔ edge ↔ services ↔ data), list assets (PII, payment data, credentials, keys) and attackers (anonymous, authenticated user, malicious insider, compromised dependency), then apply STRIDE per boundary crossing. Fixing a design flaw on paper costs minutes; in production it costs an incident.

13.2 Cryptography you must understand

Primitive	Purpose	Key facts	Use / avoid
Hash (SHA-256, SHA-3)	One-way fingerprint; **integrity**	Deterministic, fast, no key; cannot be reversed	Checksums, content ids. **Not** for passwords on its own. Avoid MD5/SHA-1
Password hash (Argon2id, bcrypt, scrypt, PBKDF2)	Store passwords	**Deliberately slow** and memory-hard, with a per-user **salt** and tunable cost	Always for passwords
HMAC	Integrity + authenticity with a shared secret	hash(key, message); verify with constant-time compare	Webhook/API signatures, signed cookies
Symmetric encryption (AES-256-GCM, ChaCha20-Poly1305)	Confidentiality of bulk data	One shared key; fast; **AEAD** modes also authenticate; never reuse a nonce	Data at rest, TLS record layer
Asymmetric (RSA, ECDSA/Ed25519, ECDH/X25519)	Key exchange and **signatures**	Public/private key pair; slow, so used to exchange a symmetric key or sign	TLS handshake, JWT signing, SSH keys, code signing
Digital signature	Authenticity + integrity + non-repudiation	Sign with private key, verify with public key	Certificates, tokens, software updates
CSPRNG	Unpredictable randomness	`crypto.randomBytes` (Node), not `Math.random()`	Session ids, tokens, salts, keys

★ **MUST REMEMBER** — **Hashing vs encryption vs encoding**

- **Hashing** is one-way (can't get the input back). **Encryption** is two-way with a key. **Encoding** (Base64, URL-encoding) is **not security at all** — anyone can reverse it. JWT payloads are Base64-encoded, *not encrypted*.

```
const argon2 = require('argon2');                       // maintained library, never hand-rolled
const hash = await argon2.hash(password, { type: argon2.argon2id });  // salt + params inside
const ok   = await argon2.verify(hash, passwordAttempt);              // constant-time verify

// Verify a webhook signature (HMAC): compare in CONSTANT time to avoid timing attacks
const expected = crypto.createHmac('sha256', secret).update(rawBody).digest('hex');
const valid = expected.length === sig.length &&
              crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(sig));
```

◆ **SENIOR INSIGHT** — **Password storage in one paragraph**

- Use **Argon2id** (or bcrypt/scrypt) with a unique salt per user and a cost tuned so one hash takes ~100–500 ms on your hardware; optionally add a server-side **pepper** held in a KMS. Never store plaintext or fast hashes (MD5, SHA-256) — attackers with GPUs try billions per second. Re-hash with stronger parameters on next login, check passwords against breached-password lists, and rate-limit login attempts (13.12).

13.3 Encrypt in transit and at rest

Your first basic rule: **encrypt in transit and at rest**. They protect against different attackers, so you need both.

	In transit (moving over a network)	At rest (stored on disk/backups)
Threat	Eavesdropping, tampering and impersonation on the network (public Wi-Fi, ISPs, internal sniffing)	Stolen disks/snapshots/backups, leaked buckets, insider or cloud-operator access, decommissioned hardware
Mechanism	**TLS 1.2/1.3** for HTTPS (Chapter 3), gRPC and DB connections; **mTLS** between services; SSH/VPN for admin access; WireGuard/IPsec tunnels	**Full-disk/volume encryption** (LUKS, EBS), **database TDE**, object-store server-side encryption (SSE-KMS), **encrypted backups**; **field-level/application-level encryption** for the most sensitive columns
Best practice	HTTPS everywhere; redirect HTTP→HTTPS; **HSTS**; TLS ≥ 1.2; modern ciphers; automate certificate renewal; validate certificates (never disable verification)	Encrypt by default (it's nearly free); separate keys per environment/tenant; encrypt snapshots and logs; protect keys more strictly than data
Doesn't protect against	Compromised endpoint, malicious server, application bugs (SQLi)	A compromised running app (it can decrypt), SQL injection, stolen credentials

Key management: where the real security lives

Encryption is only as strong as the protection of the keys. **Never store keys in source code, images, or the same place as the data.** Use a **KMS/HSM** (AWS KMS, GCP KMS, Azure Key Vault, HashiCorp Vault) and **envelope encryption**:

```
1. KMS/HSM holds the master key (KEK). It never leaves the KMS; apps call an API (audited).
2. App asks KMS: GenerateDataKey -> gets a plaintext data key (DEK) + that DEK encrypted by the master key.
3. App encrypts the data with the DEK (AES-256-GCM), stores ciphertext + encrypted DEK, and wipes the DEK.
4. To read: send the encrypted DEK to KMS (Decrypt) -> DEK -> decrypt the data.

Benefits: fast local encryption, a unique key per object/tenant, cheap master-key rotation
(re-wrap the DEKs), and a full audit trail of every key use.
```

- **Rotate** keys and credentials regularly and after any suspected leak; design systems so rotation is routine (versioned keys, re-encrypt lazily).
- **Separation of duties**: the people who administer servers shouldn't automatically be able to decrypt customer data; restrict KMS key policies and log every decrypt.

- **Sensitive data**: minimise what you store (PII, card data); **tokenise** card numbers via a payment provider (reduces PCI scope); mask in logs and UIs.

13.4 Authentication — who are you?

Authentication (AuthN) verifies identity: something you **know** (password), **have** (phone, security key) or **are** (biometrics). **Authorization (AuthZ)** — 13.5 — decides what that identity may do. Mixing them up causes major bugs (401 vs 403, Chapter 4).

Passwords, MFA and sessions

- **Passwords**: allow long passphrases (length beats complexity rules), check against breached lists, hash with Argon2id/bcrypt, throttle attempts per account and per IP, and use **generic error messages** ("invalid credentials") to avoid account enumeration.
- **MFA**: TOTP apps, push, and best of all **WebAuthn / passkeys** (phishing-resistant, public-key based). SMS is the weakest second factor.
- **Password reset**: single-use, short-lived (≈15 min), random (CSPRNG) tokens sent to the verified channel; invalidate sessions after reset.
- **Server-side session**: after login, create a random session id stored server-side (Redis, Chapter 11) and send it in a cookie with **HttpOnly** (no JavaScript access → limits XSS theft), **Secure** (HTTPS only), **SameSite=Lax/Strict** (CSRF defence). **Regenerate the id at login** (session fixation), set idle and absolute timeouts, and support instant revocation (delete the session).

```
res.cookie('sid', sessionId, { httpOnly: true, secure: true, sameSite: 'lax', maxAge: 30 * 60 * 1000 });
```

Session cookies vs tokens (JWT)

	Server-side session (opaque id)	JWT (self-contained token)
State	Stored on the server/cache; client holds only a random id	Claims + signature carried by the client; **server stays stateless** (scales easily, Chapter 5)
Revocation	**Easy**: delete the session	**Hard** until expiry: needs short lifetimes, a deny-list, or token versioning
Size / cost	Small cookie; a lookup per request	Larger header; no lookup, just signature verification
Best for	Browser web apps with first-party backend	APIs, mobile apps, service-to-service, cross-domain/SSO

```
JWT = base64url(header) . base64url(payload) . base64url(signature)
header    {"alg":"RS256","typ":"JWT"}
payload   {"sub":"user_42","iss":"https://auth.example.com","aud":"orders-api",
           "scope":"orders:read","iat":1760000000,"exp":1760000900}
signature = Sign(header + "." + payload, private key or shared secret)
NOTE: the payload is only encoded and signed - it is NOT encrypted
```

✖ COMMON MISTAKE — JWT mistakes that cause real breaches

- Accepting `alg: none` or letting the token choose the algorithm (**always pin the allowed algorithm**); not validating `iss`, `aud`, `exp`; weak HMAC secrets; putting secrets/PII in the payload; very long lifetimes; storing tokens in `localStorage` where any XSS can steal them (prefer HttpOnly, Secure, SameSite cookies for browsers). Pattern: **short-lived access token (5–15 min) + refresh token with rotation** (each refresh issues a new refresh token; reuse of an old one revokes the family).

OAuth 2.0 and OpenID Connect

OAuth 2.0 is an **authorization** framework letting an application obtain **limited access** to a user's resources on another service *without receiving the user's password* ("Sign in with Google", "allow this app to read your calendar"). **OpenID Connect (OIDC)** adds an **identity layer** on top: an **ID token** (a JWT) telling the app who logged in.

Role	Meaning
Resource owner	The user who owns the data
Client	The application requesting access (web app, mobile app, backend service)
Authorization server	Authenticates the user, obtains consent, issues tokens (Auth0, Keycloak, Okta, Cognito, Google)
Resource server	The API that validates access tokens and serves protected data

```
Authorization Code flow + PKCE (recommended for web, mobile and single-page apps)
 1. User clicks 'Log in'. Client makes a random code_verifier and sends its hash (code_challenge).
 2. Client redirects the browser to the Authorization Server:
      /authorize?response_type=code&client_id=...&redirect_uri=...&scope=openid orders:read
                &state=<random>&code_challenge=...
 3. The user authenticates (password + MFA) and consents at the Authorization Server.
 4. The server redirects back to the client with a short-lived code (+ state, which the client checks).
 5. Client (back channel, HTTPS) POSTs code + code_verifier to /token
      -> access token (+ refresh token, ID token).
 6. Client calls the API with  Authorization: Bearer <access token>;
    the API validates signature, iss, aud, exp and scope.
```

- **Grant types**: **Authorization Code + PKCE** (users), **Client Credentials** (service-to-service, no user), Device Code (TVs/CLIs). Avoid the legacy Implicit and Password grants.
- **Scopes** express coarse permissions (`orders:read`); fine-grained rules still live in your authorisation layer. Always use **state** + **PKCE** and exact-match **redirect URIs**.
- **Enterprise SSO**: SAML or OIDC with an identity provider; central MFA, de-provisioning and audit.
- **Service-to-service**: mTLS with short-lived certificates (service mesh, Chapter 8), workload identity (cloud IAM roles, SPIFFE), or client-credentials tokens — not shared static passwords.

13.5 Authorization — what may you do?

Your fourth basic rule: **use the principle of least privilege**. Authorisation must be enforced **on the server, on every request, at the object level**, with **deny by default**.

Model	How it decides	Pros / cons	Example
ACL	Per-resource list of who can do what	Simple; hard to manage at scale	File sharing permissions
RBAC (role-based)	Users → roles → permissions	Easy to reason about and audit; role explosion for fine rules	admin / editor / viewer
ABAC (attribute-based)	Policy on attributes of user, resource, action and context	Very flexible (time, department, ownership); harder to test	"Doctors may read records of their own patients during shifts"
ReBAC (relationship-based)	Permissions follow relationships in a graph (Google Zanzibar, SpiceDB, OpenFGA)	Fits sharing models (docs, folders, orgs); needs a dedicated service	"Anyone who is an editor of the folder can edit its files"

The #1 web/API vulnerability: broken access control (IDOR / BOLA)

Insecure Direct Object Reference / Broken Object-Level Authorization: the API trusts an id from the client without checking that the *caller* may access that object — changing `/orders/1001` to `/orders/1002` reveals someone else's order.

```
app.get('/orders/:id', requireAuth, async (req, res) => {
  const order = await db.getOrder(req.params.id);
  // ownership check on EVERY object access (404 avoids revealing that the id exists)
  if (!order || order.userId !== req.user.id) return res.status(404).end();
  // return only allowed fields (no mass leakage of internal columns)
  res.json(pick(order, ['id', 'status', 'items', 'total']));
});
```

- **Check at every layer that matters**: gateway (is the token valid?), service (may this user touch this object?), data (row-level security, per-tenant keys). The UI hiding a button is not authorisation.
- **Multi-tenancy**: carry `tenant_id` from the verified token into every query; enforce with DB row-level security or per-tenant schemas/databases/keys; test cross-tenant access explicitly.
- **Least privilege in practice**: DB users per service with minimal grants; cloud IAM roles scoped to specific resources/actions (no `*:*`); **short-lived credentials**; just-in-time admin access; no shared root accounts; regular access reviews.
- **Function-level checks** (BFLA): admin endpoints need role checks, not obscurity. **Property-level** (BOPLA): don't let clients set fields like `isAdmin` or `price` (mass assignment) — use allow-listed DTOs.
- Policy engines (OPA/Rego, Cedar) centralise rules so they are reviewable and testable instead of scattered `if` statements.

13.6 Input handling: injection, XSS, CSRF, SSRF

Your second and third basic rules: **sanitize all user inputs or any input parameters exposed to the user to prevent XSS and SQL injection** and **use parameterised queries to prevent SQL injection**. The root cause of every injection bug is the same: **untrusted data is interpreted as code or commands**.

SQL injection

```
// VULNERABLE - user input concatenated into the query text
db.query(`SELECT * FROM users WHERE email = '${email}'`);
//   email = "' OR '1'='1"          -> returns every user
//   email = "'; DROP TABLE users;--" -> destructive

// SAFE - parameterised query (prepared statement): the value can never become SQL
db.query('SELECT * FROM users WHERE email = $1', [email]);
```

- **Parameterised queries / prepared statements are the fix**; ORMs use them for normal queries but are **not immune** if you build raw SQL strings. Identifiers (table/column names, `ORDER BY`) can't be parameterised — use an **allow-list**.
- Defence in depth: least-privilege DB account (no DROP/DDL, read-only where possible), generic errors, WAF as a backstop (never the only control). NoSQL injection exists too (`{"$ne": null}` in a Mongo login filter) — validate types and reject operator keys.

Cross-Site Scripting (XSS)

Attacker-controlled data is rendered in a page as **executable script** (stored in your DB, reflected from a URL, or injected into the DOM). The script runs with the victim's identity: steals cookies/tokens, performs actions, defaces pages.

Defence	How
Output encoding by context	HTML-encode in HTML bodies, attribute-encode in attributes, JS-encode in scripts, URL-encode in URLs. Use frameworks that auto-escape (React, Vue, Angular, template engines); avoid `innerHTML`, `dangerouslySetInnerHTML`, `eval`

Defence	How
Content-Security-Policy	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'` — restricts where scripts may load from; blocks most injected inline scripts
HttpOnly cookies	Scripts cannot read session cookies even if XSS occurs
Validate & sanitise rich HTML	If users may submit HTML (comments, editors) use a vetted sanitiser (DOMPurify, sanitize-html) with an allow-list of tags

Other injection and request-forgery attacks

Attack	What happens	Defence
CSRF (cross-site request forgery)	A malicious site makes the victim's browser send an authenticated request (cookies attach automatically)	**SameSite** cookies; anti-CSRF tokens on state-changing requests; check Origin/Referer; don't change state on GET. (Not an issue for tokens sent in an `Authorization` header)
SSRF (server-side request forgery)	Server fetches a URL supplied by the attacker → reaches internal services or the cloud metadata endpoint (169.254.169.254) and steals credentials	Allow-list destinations, block private/link-local ranges, no redirects, IMDSv2, egress filtering, separate network
Command injection	User input reaches a shell (`exec`)	Avoid shells; use APIs with argument arrays; allow-list
Path traversal	`../../etc/passwd` in a filename	Canonicalise paths, allow-list names, store uploads by generated id outside the web root
File upload	Executable or huge or malicious files	Check type by content, size limits, store in object storage (non-executable), scan, serve from a separate domain
Insecure deserialization / prototype pollution	Crafted objects execute code or alter object prototypes (Node: `__proto__`)	Don't deserialise untrusted native objects; schema-validate JSON; freeze prototypes; update libraries
Open redirect, clickjacking	Redirect to attacker sites; framed UI tricks users	Allow-list redirect targets; `frame-ancestors`/`X-Frame-Options`
ReDoS / resource exhaustion	Catastrophic regex or giant payloads freeze the event loop (Chapter 1)	Safe regexes/timeouts, body-size limits, rate limiting

> ★ **MUST REMEMBER — Validate, encode, parameterise**
>
> • **Validate** input against an allow-list schema (type, length, range, format) at the edge; **encode** output for its context; **parameterise** anything that goes to an interpreter (SQL, shell, LDAP, XPath). Blacklists and ad-hoc "sanitising" are bypassed constantly.

13.7 OWASP Top 10 — the standard checklist

The OWASP Top 10 (listed in your notes as a source) ranks the most critical web-application risks. Edition shown: **2021** — check owasp.org for the current version.

#	Risk	Meaning	Prevention in one line
A01	**Broken access control**	Users act outside their permissions (IDOR, missing function checks)	Deny by default; server-side object-level authZ
A02	**Cryptographic failures**	Weak/no encryption, leaked keys, plaintext secrets	TLS everywhere, encrypt at rest, KMS, strong algorithms
A03	**Injection**	SQL, NoSQL, OS command, XSS	Parameterise, validate, encode

#	Risk	Meaning	Prevention in one line
A04	**Insecure design**	Missing security controls in the architecture itself	Threat modelling, secure patterns, abuse cases
A05	**Security misconfiguration**	Default creds, open buckets, verbose errors, unneeded ports	Hardened baselines, IaC scanning, least exposure
A06	**Vulnerable & outdated components**	Known-CVE libraries/OS/images	SCA scanning, patching, SBOM, minimal images
A07	**Identification & authentication failures**	Weak passwords, credential stuffing, session flaws	MFA, rate limits, secure sessions, breached-password checks
A08	**Software & data integrity failures**	Untrusted updates/CI pipelines, unsigned artifacts, insecure deserialisation	Signed builds/images, protected CI, dependency pinning
A09	**Logging & monitoring failures**	Attacks go unnoticed	Audit logs, alerting, tested incident response
A10	**Server-side request forgery**	Server fetches attacker-chosen URLs	Allow-list destinations, block internal ranges

For APIs, also read the **OWASP API Security Top 10**: broken object-level authorization (BOLA), broken authentication, broken object *property*-level authorization (excessive data exposure / mass assignment), unrestricted resource consumption (no rate limits), broken function-level authorization, unrestricted access to sensitive business flows (bots, scalping), SSRF, security misconfiguration, improper inventory management (forgotten old API versions), and unsafe consumption of third-party APIs.

13.8 API & web application security checklist

Area	Checklist
Transport	HTTPS only; HSTS; TLS ≥ 1.2; no secrets in URLs (they end up in logs); mTLS for internal APIs
AuthN / AuthZ	Standard protocols (OAuth2/OIDC), short-lived tokens, validate every claim, object- and function-level checks, deny by default
Input	Schema validation (JSON Schema/OpenAPI), body-size and field-length limits, content-type checks, reject unknown fields, parameterised queries
Output	Return only needed fields; consistent errors without stack traces or internals; encode for the context; `X-Content-Type-Options: nosniff`
Abuse protection	Rate limits and quotas per user/key/IP (Chapters 4, 14); pagination caps; request timeouts; protect expensive endpoints and business flows (login, signup, checkout, OTP) with CAPTCHA/bot detection
Browser	Cookies HttpOnly + Secure + SameSite; CSP; `frame-ancestors`; Referrer-Policy; strict CORS allow-list (CORS is browser enforcement, not server auth — Chapter 4); never reflect arbitrary Origin or use * with credentials
Webhooks & callbacks	Verify HMAC signatures with timestamp (replay protection), constant-time compare, idempotency (Chapter 12)
Lifecycle	Inventory all endpoints/versions, retire old ones, scan dependencies, automated security tests in CI (SAST/DAST/SCA), pen tests

```
const helmet = require('helmet');
app.use(helmet());                          // HSTS, nosniff, frameguard, default CSP ...
app.use(express.json({ limit: '100kb' }));  // cap the body size
app.use(rateLimit({ windowMs: 60_000, max: 100 }));  // basic throttling (Redis-backed in production)
```

13.9 Infrastructure, secrets, supply chain & DDoS

Area	Practices
Network segmentation	Public subnet only for load balancers/gateways; app and DB tiers in private subnets (Chapter 3); security groups = least-privilege allow-lists (DB accepts only the app tier); no public databases; bastion/VPN/SSM for admin access; egress filtering
Secrets management	No secrets in code, git history, container images, or plain env files. Use a secrets manager (Vault, AWS Secrets Manager/SSM, Kubernetes secrets + KMS); **short-lived, auto-rotated** credentials; separate per environment; scan repos for leaked secrets; revoke immediately when exposed
Containers & Kubernetes	Minimal base images, run as non-root, read-only root filesystem, drop capabilities, image scanning and signing, NetworkPolicies, RBAC, pod security standards, no privileged pods, resource limits
Supply chain	Pin and lock dependencies; scan for known vulnerabilities (SCA); review new packages (typosquatting, malicious maintainers); generate an SBOM; protect CI/CD (least-privilege runners, signed artifacts, protected branches, 2FA)
Patching & hardening	Automated OS/library updates, CIS baselines, immutable infrastructure (rebuild, don't patch in place), IaC scanning (Terraform/Kubernetes manifests)
WAF	Filters common attacks (SQLi/XSS patterns, bots) at the edge — a useful backstop, never a substitute for fixing the code

DDoS (denial of service) — the availability attack

Layer	Attack	Defence
L3/L4 volumetric & protocol	UDP/DNS/NTP amplification floods, SYN floods (Chapter 3)	Anycast + large scrubbing networks (CDN/cloud shield, Chapter 6), SYN cookies, upstream filtering. A 100 Gbps flood can't be fixed at your 1 Gbps origin link
L7 application	Many legitimate-looking HTTP requests; expensive endpoints; slowloris; bot traffic	CDN caching, WAF rules, per-IP/per-token rate limiting, challenges (CAPTCHA/proof-of-work), autoscaling with limits, queue back pressure and load shedding (Chapter 12)
Design-level	One request triggers lots of work (amplification inside your system)	Cap fan-out, paginate, cache, timeouts, bulkheads; protect origin so only the CDN can reach it (Chapter 6)

13.10 Logging, detection, incident response & compliance

Topic	What good looks like
Audit & security logging	Log who did what, when, from where, and the outcome (logins, permission changes, data exports, admin actions). **Never log secrets, tokens, passwords or full card numbers.** Ship to a central, append-only/immutable store with restricted access; include trace ids (Chapter 8)
Detection	Alerts on anomalies: impossible travel, spikes in 401/403/429, mass downloads, privilege changes, new admin keys, WAF hits; SIEM/SOAR; honeytokens
Incident response	Prepare (runbooks, on-call, contacts) → Detect → Contain (revoke keys, isolate hosts) → Eradicate (fix root cause) → Recover (restore from clean backups) → Post-mortem (blameless, with action items). Practise with game days
Backups & ransomware	Immutable, off-account, encrypted, versioned backups; tested restores (Chapter 2); separate credentials so an attacker who owns production cannot delete them
Privacy & compliance	**GDPR/DPDP-style** principles: data minimisation, purpose limitation, consent, retention limits, right to access/erasure, breach notification. **PCI DSS** for cards (tokenise, don't store CVV), **HIPAA** for health data, **SOC 2/ISO 27001** for organisational controls. Design deletion and data residency (geo-DNS/regional storage, Chapter 6) in from the start
Testing	Static/dynamic scanning in CI, dependency checks, regular penetration tests, bug-bounty programme, red-team exercises, chaos/security drills

13.11 Security in system design interviews

You don't need to be a security specialist, but a strong answer states **where each control sits**. Walk the request path:

Layer	Threats	Controls to mention
Edge (DNS, CDN, WAF)	DDoS, bots, scraping, injection probes	CDN/WAF, rate limiting, TLS termination + HSTS, origin lockdown
Gateway / LB	Unauthenticated access, abuse	AuthN (OAuth2/JWT validation), throttling/quotas, request size limits, mTLS to backends
Services	IDOR, injection, SSRF, confused deputy	Object-level authZ, input validation, parameterised queries, least-privilege service identities, mTLS (service mesh)
Data stores	Data theft, tampering, insider access	Encryption at rest with KMS, per-service DB roles, network isolation, field-level encryption/tokenisation, audit logging
Async / messaging	Poisoned or forged messages, replay	Authenticated producers/consumers, signed messages, schema validation, DLQ, idempotency
Operations	Leaked secrets, supply-chain compromise, slow detection	Secrets manager + rotation, signed images, dependency scanning, centralised logs, alerts, incident runbooks

Example: designing secure login for a large consumer app

1. Client sends credentials over **TLS** to the auth service (behind WAF + rate limiting: per-account and per-IP, plus bot detection for credential stuffing).
2. Auth service checks **Argon2id** hash (constant-time), applies **MFA** (TOTP/WebAuthn), and on success issues a **short-lived access token (15 min)** and a **rotating refresh token** in an HttpOnly, Secure, SameSite cookie.
3. APIs validate the JWT (pinned algorithm, `iss`/`aud`/`exp`) and enforce **object-level authorisation**; services call each other with mTLS/workload identities.
4. Suspicious logins (new device/location) trigger step-up auth and an email alert; **refresh-token reuse** revokes the whole session family.
5. All auth events go to an immutable audit log with alerts; password reset tokens are single-use, 15 minutes, and revoke other sessions.

13.12 Worked examples

Example A — Why password hashing must be slow (illustrative orders of magnitude; real numbers depend on hardware and parameters)

Item	Fast hash (SHA-256 / MD5)	bcrypt cost 12 / Argon2id tuned for ~250 ms
Attacker guess rate on a GPU	$\approx 10^9$–10^{10} guesses/s	$\approx 10^3$–10^5 guesses/s
8-char lowercase + digits ($36^8 \approx 2.8\times10^{12}$)	≈ 5–50 minutes	$\approx 10^7$–10^9 s = months to decades
Cost to you	negligible	~250 ms CPU per login: 10 logins/s ≈ 2.5 cores — plan capacity and rate-limit logins

Takeaway: slow, salted, memory-hard hashes turn a leaked database from "everyone's password in minutes" into "infeasible for strong passwords" — but weak, common passwords still fall quickly, hence breached-password checks and MFA.

Example B — Login throttling design (credential-stuffing defence)

```
Limits (token bucket / sliding window in Redis, Chapters 11 and 14):
  per account : 5 failed attempts / 15 min -> temporary lock + email notice
                (lockout can be abused to lock out a victim -> prefer progressive delays / CAPTCHA)
  per IP      : 20 attempts / min   (many users share one NAT, Chapter 3 -> combine with device / ASN signals)
  global      : alert if the overall failure rate spikes 5x over baseline -> enable CAPTCHA / bot challenge
Return the same generic error and similar response time for 'unknown user' and 'wrong password'.
```

Example C — Token lifetime trade-off

Access token 15 min: a stolen token is useful for at most 15 minutes, and revocation needs no lookup. Refresh token 30 days with rotation: if an attacker reuses an already-used refresh token, the server sees the reuse and revokes the family. Cost: one refresh call per 15 minutes per active user — for 1 M active users that is ≈ 1,100 refreshes/s (1,000,000 ÷ 900 s), trivially served by a stateless auth tier plus Redis.

Example D — Volumetric DDoS arithmetic

An attacker sends 50 Gbps of UDP junk at an origin with a 1 Gbps link: the link is saturated 50× over, so no firewall *behind* it can help. The traffic must be absorbed **upstream** — anycast CDN/scrubbing with multi-Tbps capacity — while the origin accepts connections only from the CDN's address ranges (or a private link).

13.13 Practice questions (try before looking)

P1. A user changes the URL from `/invoices/812` to `/invoices/813` and sees another customer's invoice. Name the flaw and the fix.

Answer: Broken object-level authorization (IDOR/BOLA). Check on the server that the invoice belongs to the authenticated user/tenant for every access; use unguessable ids as defence in depth, not as the control.

P2. Show how to make this safe: `db.query("SELECT * FROM t WHERE name='" + name + "'")`.

Answer: Use a parameterised query: `db.query('SELECT * FROM t WHERE name = $1', [name]);` validate input; least-privilege DB user.

P3. Where should a browser app store its tokens, and why?

Answer: Prefer HttpOnly, Secure, SameSite cookies (not readable by JavaScript, so XSS can't exfiltrate them) with CSRF protection; avoid localStorage for long-lived tokens.

P4. Why must an access token be short-lived while a refresh token can live longer?

Answer: Access tokens are sent on every request (high exposure) and hard to revoke statelessly; refresh tokens are used rarely, over a back channel, and can be rotated and revoked centrally.

P5. Your service fetches a URL provided by users (link previews). What can go wrong and how do you defend?

Answer: SSRF — attacker targets internal services or the cloud metadata endpoint. Allow-list schemes/hosts, resolve and block private/link-local IPs (re-check after redirects/DNS rebinding), isolate the fetcher in a network without internal access, use IMDSv2.

P6. What's the difference between encryption at rest and TLS, and does having both make SQL injection harmless?

Answer: TLS protects data moving on the network; at-rest encryption protects stored media. Neither stops SQL injection, because the application legitimately decrypts and reads the data — parameterisation and authZ do.

Interview Q&A (30-second answers)

Q1. Authentication vs authorization?

AuthN proves who you are (password, MFA, token); AuthZ decides what you may do (roles, attributes, ownership). Failing AuthN → 401; failing AuthZ → 403.

Q2. How do you store passwords?

Argon2id/bcrypt/scrypt with a unique salt and tuned cost, never plaintext or fast hashes; plus MFA, breach checks and login throttling.

Q3. Explain OAuth 2.0 and how it differs from OIDC.

OAuth 2.0 delegates limited API access via tokens without sharing passwords (Authorization Code + PKCE). OIDC adds an ID token so the app learns who the user is — authentication on top of OAuth.

Q4. Sessions vs JWT?

Sessions: opaque id with server-side state, easy revocation. JWT: self-contained signed token, stateless scaling, hard to revoke → short expiry + refresh rotation.

Q5. How do you prevent SQL injection and XSS?

SQLi: parameterised queries, allow-list identifiers, least-privilege DB user. XSS: context-aware output encoding, CSP, HttpOnly cookies, sanitise rich HTML.

Q6. What is CSRF and how do you stop it?

A forged cross-site request riding on the victim's cookies. Use SameSite cookies, anti-CSRF tokens, Origin checks, and no state change on GET.

Q7. How do you manage encryption keys and secrets?

KMS/HSM with envelope encryption, no keys in code or images, rotation, least-privilege access, audit logs; secrets in a manager with short-lived credentials.

Q8. What is least privilege and how do you apply it?

Grant only the minimum permissions for the minimum time: per-service DB roles, scoped IAM, short-lived credentials, JIT admin access, regular reviews.

Q9. How would you defend against DDoS?

Absorb upstream with anycast CDN/scrubbing, WAF and rate limits at the edge, cache aggressively, autoscale with limits, shed load, hide the origin.

Q10. What is zero trust?

Don't trust network location; authenticate and authorise every request and service call (mTLS, tokens), segment networks and assume breach.

AMJESH https://singhamjesh.github.io/

Chapter 13 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **The four basics:** (1) **encrypt in transit (TLS) and at rest (KMS)**; (2) **validate input + encode output** against XSS/SQLi; (3) **parameterised queries**; (4) **least privilege**.
- **CIA triad;** defence in depth; zero trust; fail closed; minimise attack surface; **STRIDE** threat modelling at design time.
- **Crypto:** hash (integrity) ≠ encryption (confidentiality) ≠ encoding (nothing). Passwords → **Argon2id/bcrypt + salt**. HMAC for signatures. AES-GCM; CSPRNG. Don't roll your own.
- **Keys:** KMS/HSM + **envelope encryption**; rotate; never in code/images; separate keys and duties.
- **AuthN:** MFA/passkeys; HttpOnly+Secure+SameSite cookies; regenerate session on login. **JWT:** pin alg, validate iss/aud/exp, 5–15 min + rotating refresh; payload is not encrypted.
- **OAuth2:** Authorization Code + **PKCE** for users, client credentials for services; scopes, state, exact redirect URIs. **OIDC** = ID token (identity).
- **AuthZ:** server-side, per object, deny by default (RBAC/ABAC/ReBAC). #1 bug = **IDOR/BOLA**. Multi-tenant: tenant id everywhere.
- **Input attacks:** SQLi → parameterise; XSS → encode + CSP; CSRF → SameSite + tokens; SSRF → allow-list + block internal IPs; validate with allow-lists.
- **OWASP Top 10:** access control, crypto failures, injection, insecure design, misconfig, vulnerable components, auth failures, integrity failures, logging gaps, SSRF.
- **Infra:** private subnets, least-privilege security groups, secrets manager, non-root containers, signed images, dependency scanning, WAF as backstop, DDoS absorbed upstream.
- **Operate:** audit logs (never secrets), alerts, incident runbooks, immutable tested backups, compliance (GDPR/PCI) by design.

AMJESH https://singhamjesh.github.io/